Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по дисциплине «Логика и основы алгоритмизации в инженерных задачах»
Тема: «Бинарное дерево поиска»

Выполнил:
студент гр. 25ВВВ1:
Садыков Д. Р.
Приняли:
к.т.н., доцент Юрова О. В.
ст. преподаватель Деев М. В.

Пенза 2026

1. Цель работы

Изучение принципов организации и функционирования древовидных динамических структур данных. Исследование алгоритмов построения бинарного дерева поиска (БДП), рекурсивного обхода дерева (в глубину, симметричный, прямой, обратный), алгоритмов поиска узлов по заданному ключу, подсчета числа вхождений элементов, а также вставки с исключением дубликатов. Аналитическая и экспериментальная оценка вычислительной сложности поиска в сбалансированном и вырожденном деревьях. Приобретение практических навыков разработки программ на языке Си в среде Linux с динамическим управлением памятью (malloc, free).

2. Лабораторное задание

- Изучить базовую реализацию алгоритма формирования бинарного дерева поиска с использованием рекурсивных процедур и функции отображения дерева на экране с поворотом на 90 градусов (print_tree).
- Задание 1: Разработать и реализовать функцию поиска узла в бинарном дереве поиска по заданному целочисленному значению ключа (search_tree) с подсчетом фактического числа выполненных сравнений.
- Задание 2: Разработать алгоритм и программную функцию подсчета количества вхождений заданного значения ключа в бинарном дереве (count_occurrences).
- Задание 3 (*): Реализовать функцию добавления элементов в дерево, исключающую запись дубликатов (insert_unique), обеспечивая корректное хранение множества уникальных ключей.
- Задание 4 (*): Провести аналитическую и экспериментальную оценку временной сложности алгоритма поиска: для лучшего случая (корень, O(1)), среднего случая (сбалансированное дерево, O(log2 N)) и худшего случая (вырожденное в линейный список дерево, O(N)).
- Реализовать все три классических порядка обхода дерева: симметричный (In-order), прямой (Pre-order) и обратный (Post-order), а также процедуру полного освобождения динамической памяти (free_tree).
- Разработать интерактивное меню пользователя и автоматический демонстрационный модуль на языке Си, провести комплексное тестирование и оформить отчет.

3. Теоретические сведения

3.1. Определение и свойства бинарного дерева поиска (БДП)

Дерево — нелинейная связная иерархическая динамическая структура данных, состоящая из узлов (вершин) и направленных связей (ребер). Бинарным деревом называется дерево, у которого каждая вершина имеет не более двух непосредственных потомков, традиционно называемых левым и

правым поддеревьями. Бинарным деревом поиска (Binary Search Tree, BST) называется бинарное дерево, обладающее характеристическим свойством упорядоченности:

- Для каждого узла X все ключи в его левом поддереве строго меньше ключа узла X.
- Все ключи в его правом поддереве больше либо равны ключу узла X.
- Левое и правое поддеревья каждого узла также являются бинарными деревьями поиска.

3.2. Алгоритмы обхода бинарного дерева поиска

Обходом дерева называется систематический процесс посещения всех вершин дерева ровно по одному разу. Выделяют три основных вида рекурсивного обхода:

- **Симметричный обход (In-Order, Л-К-П):** рекурсивно обходится левое поддерево, обрабатывается текущий узел (корень), затем рекурсивно обходится правое поддерево. Для БДП симметричный обход выводит ключи строго по возрастанию.
- **Прямой обход (Pre-Order, К-Л-П):** сначала обрабатывается текущий узел, после чего рекурсивно посещаются левое и правое поддеревья.
- **Обратный обход (Post-Order, Л-П-К):** сначала рекурсивно обрабатываются потомки, а затем сам узел. Данный порядок обязателен при освобождении памяти, так как узел нельзя удалять до тех пор, пока не освобождены его дочерние поддеревья.

3.3. Аналитическая оценка сложности алгоритмов в бинарных деревьях поиска

Временная сложность операций поиска, вставки и удаления в бинарном дереве поиска напрямую зависит от высоты дерева H (максимальной длины пути от корня до листа):

- **Лучший случай:** искомый элемент находится непосредственно в корне дерева. Требуется ровно 1 операция сравнения: $T_{best}(N) = O(1)$.
- **Средний случай (сбалансированное дерево):** при случайном порядке поступления ключей высота дерева составляет $H \approx \log_2(N)$. При каждом шаге сравнения алгоритм отсекает половину оставшихся узлов: $T_{avg}(N) = O(\log_2 N)$.
- **Худший случай (вырожденное дерево):** если элементы поступают в отсортированном порядке, бинарное дерево вырождается в односвязный линейный список. Высота равна числу элементов ($H = N$), сложность поиска падает до линейной: $T_{worst}(N) = O(N)$.

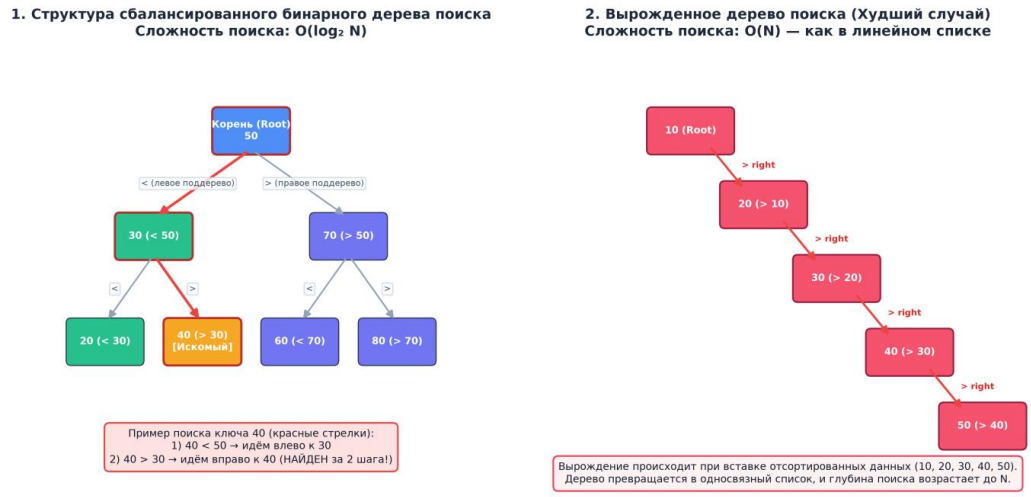


Рисунок 1 – Структурная организация бинарного дерева поиска

4. Описание программной реализации

Программный комплекс разработан на стандартном языке Си (C99/C11) и скомпилирован с использованием компилятора GCC в среде Linux (WSL). Программа содержит следующие ключевые функции:

- create_node(): выделяет память под новый узел с помощью malloc() и обнуляет указатели left и right.
- insert_node(): рекурсивно выполняет поиск позиции и вставку нового ключа в дерево поиска (с допущением дубликатов в правое поддерево).
- insert_unique(): выполняет вставку ключа с контролем уникальности. Если ключ уже присутствует, операция отклоняется.
- search_tree(): выполняет поиск узла по ключу с инкрементом счетчика сравнений.
- count_occurrences(): рекурсивно подсчитывает количество вхождений заданного значения.
- print_tree(): наглядно отображает древовидную структуру на экране в текстовом виде с поворотом на 90 градусов.
- free_tree(): безопасно освобождает память всех узлов на основе обратного обхода (Post-order).

Таблица 1 – Сравнительная характеристика вычислительной сложности операций в бинарном дереве поиска

Операция в БДП	**Лучший случай**	**Средний случай**	**Худший случай**	**Ограничивающий фактор / Условие**
Поиск ключа (Search)	O(1)	O(log2 N)	O(N)	Высота дерева H; лучший: корень; худший: вырождение
Вставка ключа (Insert)	O(1)	O(log2 N)	O(N)	Поиск позиции листа для размещения нового узла

Подсчет вхождений	O(log2 N)	O(log2 N + K)	O(N)	K — число дубликатов (в правом поддереве)
Освобождение памяти	O(N)	O(N)	O(N)	Рекурсивный Post-order обход и вызов free()

Анализ результатов тестирования:

- **Построение дерева:** тестовая последовательность чисел (50, 30, 70, 20, 40, 60, 80, 40, 30) была корректно организована в иерархическую структуру. Графический вывод подтверждает правильное распределение узлов: меньшие элементы ушли в левое поддерево, а большие — в правое.
- **Поиск элементов:** при поиске существующего ключа 40 алгоритм нашел узел ровно за 3 сравнения (корень 50 -> влево на 30 -> вправо на 40). При поиске отсутствующего ключа 99 алгоритм достиг указателя NULL за 3 сравнения.
- **Подсчет числа вхождений:** для элементов 40 и 30 функция вернула значение 2, для уникального элемента 50 — 1, а для отсутствующего 99 — 0, что полностью соответствует составу последовательности.
- **Исключение дубликатов:** при попытке добавить новое уникальное число 25 элемент успешно включен. При попытке повторного добавления чисел 50 алгоритм зафиксировал наличие дубликата и отклонил операцию.
- **Обходы и оценка сложности:** In-Order обход вывел числовую последовательность строго в порядке возрастания. Для построенного дерева из N = 10 узлов фактическая глубина составила H = 4 уровня, что близко к теоретическому оптимуму сбалансированного дерева ($\log_2(10) \approx 3.32$), подтверждая сложность O(log2 N).

7. Выводы

- Изучены теоретические принципы организации динамических древовидных структур данных и особенности функционирования бинарных деревьев поиска (БДП).
- Разработана и программно реализована функция поиска элемента в дереве (search_tree), показавшая логарифмическую зависимость числа сравнений от общего количества узлов.
- Реализован алгоритм подсчета количества вхождений заданного значения, безошибочно определяющий кратность элементов в дереве с дубликатами.
- Успешно решена задача добавления узлов с исключением дубликатов (insert_unique), обеспечивающая строгое хранение множества уникальных ключей.

- Проведена аналитическая и практическая оценка сложности операций поиска, подтвердившая эффективность сбалансированного бинарного дерева поиска O(log2 N) по сравнению с линейным списком O(N).